Backend Engineering ≠ Springboot or Java

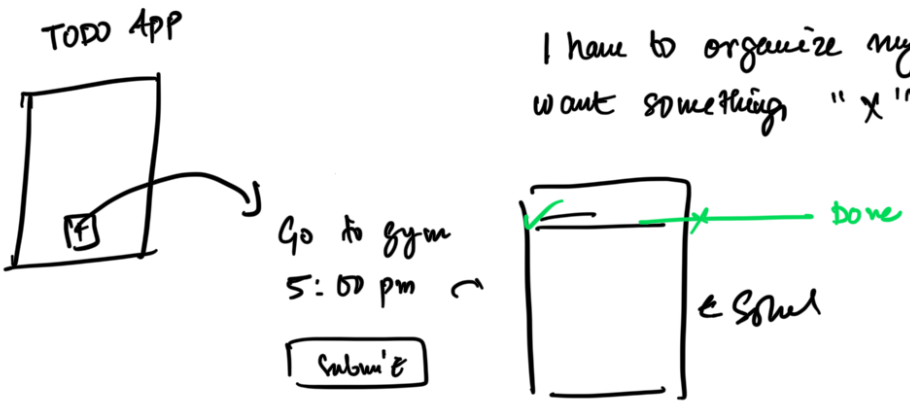
① It is not about language

i) Design Backend Systems
ii) Design Database
iii) API Design
iv) Distributed Systems
v) Microservices or Monolith
vi) Deployment (CI/CD, Cloud, K8, Containers)
vii) Scaling

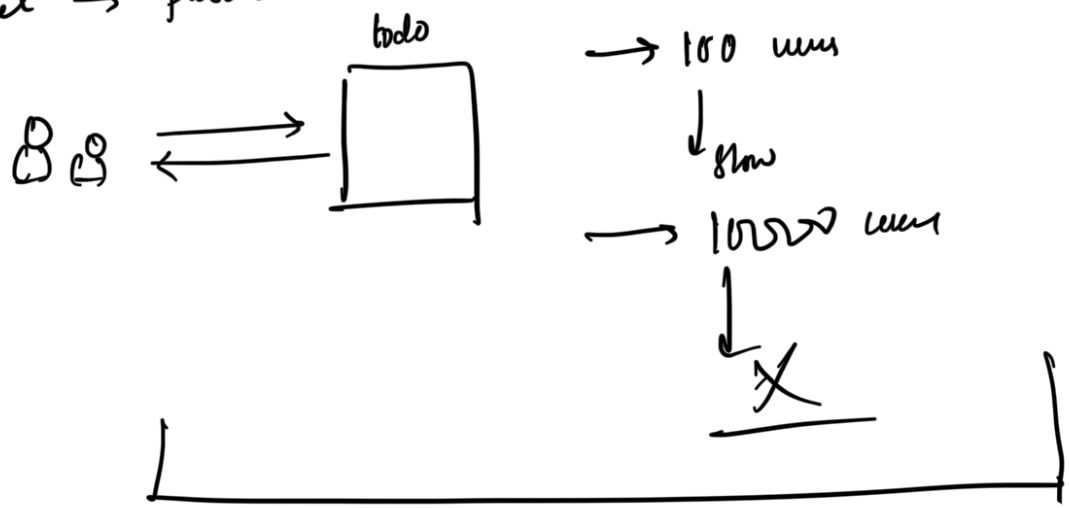
Example

① Problem Statement

I have to organize my tasks i' want something "X".



Schedule → publish



"How did you design your backend?"

1 user, 100 user — 1M users → by backend will able to handle.

↳ System design of backend engg.

* Enterprise level backends. & System.

1) How do you start?

Scenarios

automatically

you work in a company

you'r an starting a startup

already backend code

- infra
- language
- architecture

Product

Ani

KT

Ideation

Create Backend design

chapter

Final Assignment.

Backend project

→ design architecture
→ database design
→ API design
→ infra design
→ Language

team project

Projects

* AirBnB backend
  ↳ monolith
* LinkedIn backend
  ↳ microservice
* Lovable backend
* Uber's backend

Spotify Backend.

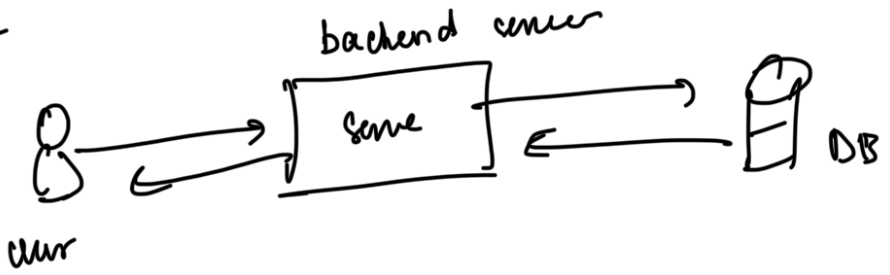
Backend Engineering.

(i) problem statement (ii) Choose database (iii) Entity design

note pros and cons .... (iv) API design (v) Deployment

(System Design)

HLD



Java → language

→ Backend APIs
→ Enterprise Soft
→ Banking system
→ Android Apps
→ Distributed Systems.

* IntelliJ IDEA
* postgreSQL
* MySQL
* postman / Insomnia.

✓ platform independent → "Write once, run anywhere".

Java code → web, mobile, IOT

Java → JVM (Java Virtual Machine)

✓ Object Oriented language

Every be functioning thing is an object in Java

Functional programming — OOPS

Class & Objects

Amol → Creta (car) → object

✓ torment, ✓ drive it ✓ washit

car → Object

brand → hyundai
color → black
speed → 150

} properties / attributes

start, stop, accelerate } actions / methods

---

blueprint of how my student work

→ name
→ age
→ rollNumber

study
attend class

```
class Student {
    String name;
    int age;
    int rollnumber;

    void study() {
        S.out (name + " is studying");
    }
}
```

Student amol = new Student(); → Object

Student stud = new Student(); → objects

An object is a real instance of a class that contains data (properties) and behavior (methods)